12

Reasoning with GRPO: The DeepSeek Recipe

The Paradigm Shift: Skipping SFT

The standard pipeline for building a capable LLM has three stages: pretrain on text, supervised fine-tune on demonstrations (Chapter 5), then align with RL (Chapters 6–10). Every major model through 2023–ChatGPT, Claude, Llama-Chat–followed this recipe. Supervised fine-tuning was considered essential: you needed to show the model thousands of examples of good behavior before RL could refine it further.

In early 2024, DeepSeek challenged this assumption. Their model, DeepSeek-R1, achieved state-of-the-art reasoning performance with a radically simplified pipeline: **Pretrain → RL**. That is it. No supervised fine-tuning. No human-written demonstrations of step-by-step reasoning. Just a pretrained model, a reward signal ("did you get the right answer?"), and GRPO.

The result was not merely competitive with SFT-based approaches–it surpassed them. DeepSeek-R1 discovered reasoning strategies that no human had explicitly taught it, including elaborate self-verification, multi-path

problem solving, and spontaneous chain-of-thought. The model figured out that "showing its work" leads to better final answers, entirely on its own.

Why does skipping SFT work? Supervised fine-tuning teaches by imitation: the model sees thousands of correct solutions and learns to reproduce their patterns. This is effective but inherently bounded–the model can only be as good as the examples it trains on, and it learns surface patterns as much as deep understanding. RL, by contrast, teaches by exploration: the model generates its own solutions, receives feedback (correct or incorrect), and discovers strategies that maximize reward. There is no ceiling imposed by human examples because the model is free to find approaches humans would not have written.

The risk is obvious: without SFT to provide a "warm start," the model begins RL from a pretrained checkpoint that can generate text but has no concept of instruction following, structured reasoning, or helpful behavior. The early stages of training are chaotic. But DeepSeek showed that with enough compute and the right algorithm, the model bootstraps these capabilities from scratch.

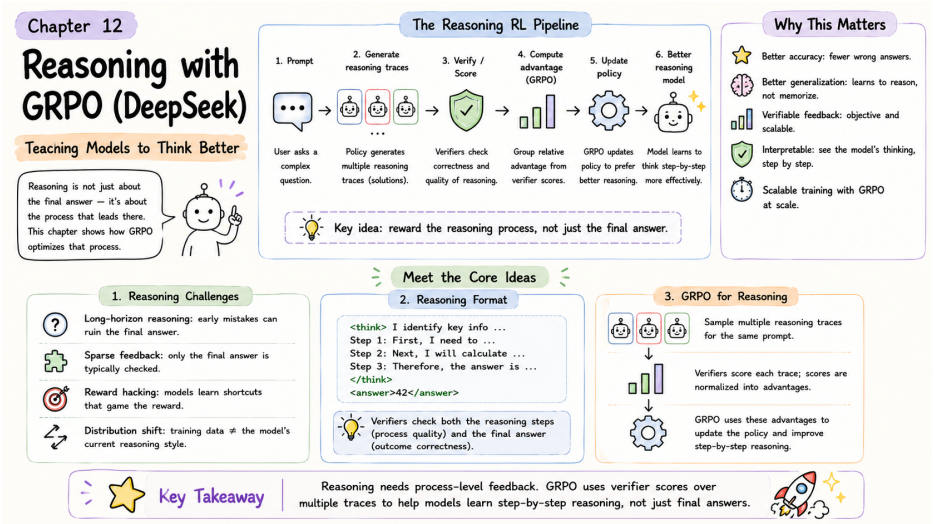

Figure 12.1: Reasoning with GRPO at a glance – reward correctness and let the model discover chain-of-thought strategies on its own.

The DeepSeek Recipe

The recipe has three ingredients: the GRPO algorithm (Chapter 10), simple binary verifiers (Chapter 11), and a curriculum that progresses from easy to hard problems.

GRPO as the Training Algorithm

Chapter 10 derived GRPO's objective function and walked through its mechanics: generate a group of responses per prompt, score each with a reward function, compute normalized advantages using the group mean and standard deviation, and update the policy with PPO-style clipping and a KL penalty.

DeepSeek's application of GRPO used groups of $G = 8$ to 16 responses per prompt. The reward was binary: +1 for a correct final answer, 0 for an incorrect one. No reward model was trained. No human preferences were collected. The entire reward signal came from deterministic verifiers–math checkers that compared the model's answer against ground truth, and code execution environments that ran the model's programs against test cases.

Why binary rewards are enough.

It seems counterintuitive that a reward of just 0 or 1 can teach sophisticated reasoning. The insight is that GRPO does not need rich reward signals–it needs *variance within each group*. If 3 out of 8 responses are correct and 5 are wrong, the correct ones get positive advantage and the wrong ones get negative advantage. The model then asks: "What did the correct responses do differently?" Over millions of problems and groups, the pattern that emerges is consistent: correct responses tend to show their work, break problems into steps, and verify intermediate results. The model reinforces these behaviors without anyone explicitly defining or rewarding them.

Curriculum Learning: Easy to Hard

DeepSeek did not start with competition-level mathematics. The training curriculum progressed through difficulty levels, allowing the model to build foundational skills before

tackling hard problems.

Stage 1: Arithmetic and basic algebra. Problems like "What is $5 + 7$?" or "Solve $2x = 10$." Clear right/wrong answers, fast feedback, high success rates. The model learns that attempting calculation leads to higher reward than generating random text.

Stage 2: Multi-step word problems. "If John has 5 apples and buys 3 more, then gives 2 away, how many does he have?" Requires planning and tracking intermediate results. The model learns that breaking problems into steps is beneficial.

Stage 3: Competition-level math and complex code. Problems that require sophisticated reasoning, creative approaches, and multi-step proofs. Only works if the earlier stages established basic reasoning patterns.

Why curriculum matters. Without curriculum, the model faces hard problems from the start and gets reward 0 on nearly every attempt. Every response in the group is wrong, so $\sigma_G = 0$ and GRPO produces no gradient. The model learns nothing. Curriculum ensures that at every stage, there is enough variance within groups to produce useful learning signals. Easy problems have high variance early (some correct, some not); hard problems develop variance later as the model's skills improve.

Compute Cost Analysis

Standard pretraining processes each example with one forward pass. GRPO-enhanced training generates G complete responses per prompt, evaluates each with a verifier, and computes a backward pass.

$$\text{Cost multiplier} \approx \frac{G \times (C_{\text{forward}} + C_{\text{reward}} + C_{\text{backward}})}{C_{\text{forward}}} \approx 2\text{–}4\times$$

With $G = 8$ and cheap verifiers (math checking is nearly free compared to neural network inference), the practical overhead is roughly 3× standard pretraining cost. DeepSeek

considered this worthwhile: the resulting model has emergent reasoning abilities that would otherwise require expensive human-annotated reasoning traces for SFT, discovers strategies humans would not have taught, and scales predictably with more compute.

The Emergence of Chain-of-Thought

The most surprising outcome of the DeepSeek recipe was not the accuracy numbers but *how* the model achieved them. Nobody told DeepSeek-R1 to think step by step. Nobody showed it examples of chain-of-thought reasoning. Yet it discovered this strategy on its own.

Why RL Discovers Reasoning

Two forces compete during GRPO training.

Force 1: Maximize reward. Longer, more detailed responses tend to produce fewer mistakes and higher accuracy on the final answer. This creates an incentive to generate elaborate reasoning chains.

Force 2: Minimize length (implicit). Each token has probability less than 1. Longer sequences have lower total probability: $p(\text{long}) = p(t_1) \cdot p(t_2) \cdots p(t_{100})$. This creates an implicit incentive toward shorter responses.

The critical insight is that **RL does not see absolute probabilities**. The policy gradient operates on rewards, not likelihoods. When the model generates a short response and gets reward 0, it learns to avoid that pattern. When it generates a long, detailed response and gets reward 1, it learns to do more of that. The reward boost from correct answers overwhelms the probability penalty from extra length.

The length-reward trade-off, quantified.

Consider a model solving a math problem with two strategies:

Strategy 1: Direct answer. "The answer is 42." Accuracy: 40%.

Strategy 2: Step-by-step reasoning. "Let me break this down… [50 tokens of reasoning]… Therefore, the answer is 42." Accuracy: 85%.

From RL's perspective, Strategy 1 produces reward 0 sixty percent of the time. Strategy 2 produces reward 1 eighty-five percent of the time. The policy gradient strongly reinforces Strategy 2. Over many training steps, the model shifts probability mass from short, unreliable responses toward longer, more reliable reasoning chains. The key is that RL sees the *observed* rewards–not the prior probability of generating each strategy. A strategy that was initially improbable but consistently gets high rewards will be strongly reinforced until it becomes the model's default behavior.

What the Model Discovers

Emergent reasoning patterns in DeepSeek-R1.

As training progresses, the model spontaneously developed several reasoning patterns that were never explicitly taught:

Showing intermediate steps: Writing out each algebraic manipulation rather than jumping to the answer. This emerges because intermediate steps make arithmetic errors less likely.

Self-verification: Checking its own work, sometimes with phrases like "Wait, let me verify this…" This emerges because catching errors before the final answer increases accuracy.

Multiple solution paths: Solving a problem two different ways and comparing results. This emerges because agreement between two methods strongly predicts correctness–a strategy few humans would have explicitly demonstrated in training data.

Error recovery: Recognizing a mistake mid-solution and backtracking to try a different approach. This emerges because models that can recover from dead ends get more problems right than models that commit to their first approach.

All of these emerged because they increase the probability of correct final

answers, and GRPO reinforces whatever leads to higher reward.

Training Phases and the "Aha!" Moment

DeepSeek-R1's training showed four distinct phases, with a sharp phase transition between phases 2 and 3. Understanding these phases is important for practitioners because it sets expectations: the model will appear to make no progress for thousands of steps before a sudden breakthrough.

Phase 1: Chaos (Steps 0–1,000). Model outputs are incoherent–random text with no mathematical structure. Rewards are near zero on all but the simplest problems. The model is exploring randomly. This phase is discouraging but unavoidable: the model must generate enough diverse outputs to stumble on patterns that work.

Phase 2: Structure Discovery (Steps 1,000–5,000). The model learns to format outputs as problem-then-solution. It begins attempting calculations and produces recognizable (though often wrong) mathematical expressions. Accuracy reaches 10–20%. The model has learned *what kind of thing* to output but not *how* to get it right. Responses are short–typically one or two lines–because the model has not yet discovered that longer reasoning helps.

Phase 3: The "Aha!" Moment (Steps 5,000–8,000). This is the critical phase transition. Intermediate reasoning steps appear *suddenly*–not through gradual lengthening of responses but as an abrupt qualitative change. Accuracy jumps from roughly 20% to 55% over a few hundred steps. Average response length doubles or triples. The model has discovered the length-quality correlation: detailed reasoning leads to correct answers.

Phase 4: Refinement (Steps 8,000+). Reasoning becomes more sophisticated. Self-correction appears ("Wait, that doesn't seem right…"). Multiple solution strategies emerge for the same problem type. Accuracy continues climbing from 55% toward 85%+, but the improvement is now gradual rather than sudden. The model is optimizing *how* to reason, not discovering *that* reasoning helps.

Why the "aha!" is sudden, not gradual.

Think about learning to ride a bicycle. Days 1 through 10: constant falling, no visible progress. Day 11: suddenly you can balance. Days 12 through 20: rapid improvement in speed and control. There is a critical threshold where the skill "clicks."

RL training shows the same phase transition dynamics.

Before the aha: The model tries short responses, gets low rewards, incrementally tries slightly longer ones, still gets low rewards because the reasoning is not yet coherent enough to improve accuracy. There is no gradient signal favoring length, because incoherent length does not help.

At the aha: The model generates its first coherent multi-step reasoning chain–possibly by chance–and gets a high reward. The policy gradient strongly reinforces this pattern. Crucially, all similar reasoning structures get boosted simultaneously, because they share token-level patterns ("Step 1:", "Therefore,", "Let me check…"). This creates a cascade: one successful reasoning chain unlocks many, because the model learns to produce the *structural markers* of reasoning, and those markers improve accuracy across many different problems at once.

After the aha: The model now "knows" that reasoning helps and explores how to reason *better*–more steps, cleaner logic, self-verification. Improvement becomes steady rather than explosive.

Implications for practitioners. If you are training with GRPO and see no improvement for thousands of steps, do not panic. The "aha!" moment is real and well-documented. Monitor average response length alongside accuracy: a sudden increase in response length (without explicit encouragement) is a

leading indicator that the phase transition is beginning. If response length remains flat after 10,000+ steps, something is wrong–check your reward signal, group size, and curriculum difficulty.

Three Types of Chain-of-Thought

DeepSeek-R1's RL-discovered reasoning is one of three ways to get chain-of-thought behavior from an LLM. Understanding the alternatives clarifies what makes the RL approach special.

Type	How Created	Advantages	Limitations
Prompted CoT	Add "Let's think step by step" to the prompt	Zero training cost, works immediately, interpretable	Quality depends on base model, not task-optimized
Fine-tuned CoT	SFT on human-written reasoning traces	High quality, consistent style	Expensive data, bounded by human examples
RL-discovered CoT	Emerges from pure reward signal (GRPO)	Best performance, discovers novel strategies, task-optimized	High training cost (one-time), less interpretable

Performance on difficult reasoning tasks follows a consistent ranking:

Method	Typical Accuracy	Training Cost
No chain-of-thought	30–50%	None
Prompted CoT	65–75%	None (inference cost only)
Fine-tuned CoT	75–85%	Medium (annotation + SFT)
RL-discovered CoT	85–90%	High (one-time RL training)

Why RL-discovered CoT wins. Fine-tuned CoT is constrained by human intuitions about how to solve problems. Humans write solutions in a particular style, use familiar

strategies, and follow conventions. RL-discovered CoT has no such constraints. It discovers whatever reasoning patterns actually maximize reward, including unconventional ones. Models trained with GRPO often discover that solving a problem multiple ways and comparing results is highly effective–a meta-strategy that few humans would have explicitly demonstrated in training data.

The interpretability trade-off. RL-discovered reasoning is often *less* interpretable than human-written chains. The model may take detours, use unusual notation, or arrive at correct answers through reasoning paths that are hard for humans to follow. This is fine when correctness is verifiable (math, code), but can be problematic for tasks where humans need to audit the reasoning (medical diagnosis, legal analysis). For such tasks, fine-tuned CoT with human-written traces may be preferable despite lower peak performance.

Cost comparison at scale. Prompted CoT costs roughly 7–8× more per query than no-CoT (150 tokens vs 20 tokens). RL-discovered CoT has the same per-query cost as prompted CoT, but adds a one-time training cost of \$100K–\$1M. At 100 million queries, the training cost amortizes to less than \$0.01 per query–negligible compared to inference costs. For high-volume applications, RL-discovered CoT is the most cost-effective approach.

Scaling Laws for Reasoning

Reasoning ability scales differently from standard language modeling. In pretraining, performance improves as a slow power law: doubling compute produces a modest reduction in loss. In RL training for reasoning, the scaling is steeper.

Scaling Comparison

Standard pretraining: Loss $\propto C^{-\alpha}$ where $\alpha \approx 0.05$–0.10. Doubling compute reduces loss by 3–7%.

RL for reasoning: Accuracy $\propto C^{-\beta}$ where $\beta \approx 0.15$–0.25. Doubling compute improves accuracy by 10–18%.

The difference arises because reasoning is a *skill* that can be learned through practice. More compute means more exploration, more groups, and more opportunities to discover and reinforce effective strategies. Phase transitions (the "aha!" moment) create non-smooth scaling jumps that amplify gains at certain compute thresholds.

Compute sweet spot analysis:

RL Compute	MATH Accuracy	Cost	Gain per Dollar
1× (baseline)	45%	\$10K	–
10×	68%	\$100K	0.26 points / \$1K
100×	82%	\$1M	0.016 points / \$1K
1,000×	89%	\$10M	0.0008 points / \$1K

The 1× to 10× range delivers the best value: a 23-point accuracy gain for \$90K in additional compute. The 10× to 100× range still produces meaningful gains (14 points) but at 6× lower efficiency per dollar. Beyond 100×, diminishing returns set in sharply. For most practical applications, 10–100× compute is the sweet spot.

Practical Implications

When Pure RL Works

The DeepSeek recipe–skip SFT, train with GRPO and verifiers–works well under specific conditions.

Objective rewards. The task must have a deterministic or near-deterministic way to check correctness: math verification, code execution, formal proofs, factual question answering with known answers. Without reliable rewards, GRPO's group baselines are noise.

Sufficient compute. The chaotic early phases (Steps 0–5,000) require patience and com-

pute. The model burns through thousands of training steps producing garbage before the "aha!" moment. Smaller teams may not be able to afford this exploration phase.

Novel strategy discovery matters. Pure RL shines when you *want* the model to discover approaches humans have not thought of. For competition math, code generation, and theorem proving, this is a significant advantage. For tasks where you need predictable, human-readable reasoning, the RL approach may be less suitable.

When SFT Still Helps

For many practical applications, the "best of both worlds" approach is preferable: minimal SFT for basic instruction following, then aggressive RL for capability development.

Subjective tasks. Creative writing, style matching, tone control–there is no verifier for "is this essay well-written?" These tasks need human preferences (Chapters 6–7) or at minimum some SFT demonstrations to define what "good" looks like.

Limited compute. If you cannot afford the chaotic early phases of pure RL, SFT provides a warm start that gets the model to a reasonable baseline quickly, and RL refines from there. This is the practical choice for most teams.

Specific output formats. If the model must produce JSON, follow a template, or adhere to length constraints, SFT is the most direct way to teach these structural requirements. RL can reinforce them afterward but may struggle to discover them from scratch.

Cold-start scenarios. A model that has never seen instruction-following examples may take far longer to converge with pure RL than one that starts from an SFT checkpoint. The SFT stage dramatically reduces the time spent in Phase 1 (chaos) by giving the model a head start on structured output generation.

DeepSeek-R1 vs OpenAI o1

Both DeepSeek-R1 and OpenAI's o1 represent a broader trend: investing computation into reasoning rather than just knowledge. But they take different approaches.

Dimension	DeepSeek-R1	OpenAI o1
Training approach	Pure RL with GRPO (no SFT)	Likely SFT → RLHF (details undisclosed)
Public details	Open weights, published methodology	Closed, limited information
Reasoning style	Explicit, visible chain-of-thought	Hidden "thinking" tokens
Training cost	Lower (simpler pipeline)	Higher (multi-stage)
Performance	State-of-the-art reasoning	State-of-the-art reasoning
Key contribution	Proved SFT is unnecessary for reasoning	Demonstrated test-time compute scaling

Both models demonstrate a fundamental insight: investing computation into learning *how to reason*–whether at training time (DeepSeek's approach with GRPO) or inference time (o1's approach with test-time compute, covered in Chapter 13)–produces models that can reason rather than merely pattern-match. DeepSeek-R1's specific contribution is showing that the expensive human annotation step (writing thousands of step-by-step solutions for SFT) can be eliminated entirely. The model discovers reasoning from reward alone.

The DeepSeek recipe in context.

Algorithm: GRPO with groups of 8–16 (Chapter 10). No critic, no reward model–just the policy, a reference model, and a verifier.

Reward: Binary correct/incorrect from deterministic verifiers (Chapter 11). No learned scorer to hack.

Curriculum: Easy → medium → hard problems. Ensures useful variance

in GRPO groups at every stage.

What emerges: Chain-of-thought reasoning, self-verification, multi-path problem solving, error recovery–all without a single human demonstration.

What it costs: Roughly 3× standard pretraining compute. No annotation cost.

Chapter 13 covers what happens *after* training: how test-time compute techniques (best-of-N, beam search, MCTS) can further improve a GRPO-trained model's performance at inference time by spending extra computation per query.

The companion code for this chapter is at `github.com/arunpshankar/packt-final` – swap `github.com` for `githubtocolab.com` to open it directly in Colab.